ADA University
School of Information Technologies and Engineering
CSCI 3509: Intro to Software Engineering
Fall 2026

Project Proposal

Q□na□t: Smart Personal Finance & Micro-Savings Platform

Project team: Q□na□t Engineering Team

Instructor: Kamila Ismayilova

Submitted in partial fulfillment of the requirements of the CSCI 3509: Intro to Software Engineering course project

Team member	Contribution to the homework
Ali Baghirov	Project lead and full-stack software engineer: Designed system architecture and REST API specifications; implemented the dynamic micro-savings algorithm, exact decimal financial math, and open banking transaction deduplication; integrated OTP/FIN authentication, habit review gamification streak engine, and monthly reporting analytics with CSV export.

1. Project Overview

Q□na□t is a localized smart personal finance and micro-savings web platform developed specifically for the Azerbaijani market (AZN). In everyday financial management, individuals often lack clarity on how much discretionary money they can safely spend after accounting for upcoming commitments, while impulsive withdrawals frequently sabotage long-term savings intentions. Q□na□t eliminates this friction by answering five immediate daily financial questions at a glance, connecting to local open banking APIs with automated transaction deduplication, and dynamically calculating exact daily saving contributions toward user-defined goals. To safeguard financial commitments, the platform introduces an anti-impulse cooling-off safety lock on withdrawals and reinforces daily discipline through a habit-building gamification streak engine.

Primary Users:

- **Everyday Savers and Young Professionals:** Need an intuitive, AZN-denominated expense tracker that imports card transactions from local banks and categorizes spending without tedious manual ledger entry.
- **Goal-Oriented Individuals and Students:** Need structured, algorithmic guidance on how much to deposit daily toward specific life milestones (e.g., tech purchases, emergency fund, tuition) paired with withdrawal cooling-off locks to curb impulse spending.
- **Household Budgeters and Financial Planners:** Need comprehensive month-over-month cash flow summaries, spending breakdowns by category, multi-user workspace switching, and authenticated CSV data exports.

Main System Features:

- **5-Question Instant Financial Dashboard:** Delivers instant clarity on total account balance, unallocated available cash, monthly earnings, monthly expenses, and net cash flow health.
- **Dynamic Daily Savings Recommendation Engine:** Automatically computes the exact daily contribution required for each goal based on target amount, accumulated balance, and days remaining until deadline.
- **Anti-Impulse Savings Security Lock (Soyuma Müdd□ti):** Prevents spontaneous dipping into savings goals by enforcing an intentional cooling-off waiting period before funds can be withdrawn.
- **Open Banking Integration with Automated Deduplication:** Ingests transaction feeds from Azerbaijani banks (Kapital Bank, ABB, Leo, CBAR sandbox) and filters duplicate incoming records.
- **Discipline Gamification & Review Streak Tracker:** Motivates consistent daily financial tracking by awarding consecutive day streak counters, milestone badges (10, 20, 30, 40, 100 days), and experience levels.
- **Comparative Monthly Analytics & CSV Export:** Displays month-over-month variances in revenue and expenses alongside visual category distribution charts, with one-click authenticated CSV download.
- **Secure Multi-Factor Authentication:** Authenticates users through mobile phone numbers, national FİN codes, and time-expiring 6-digit SMS OTP verification.

2. Tech Stack

The system architecture strictly decouples the client-side presentation layer from the server-side REST API, leverages structured persistence with atomic integrity, and integrates machine learning intelligence with automated real-time operational alerting.

Layer	Technology	Why we chose it
Frontend (UI)	React 19, TypeScript, Tailwind CSS, Lucide Icons	Provides a lightning-fast, reactive component architecture with strict compile-time type safety; Tailwind CSS enables an ergonomic, high-density dark fintech UI tailored for clarity and mobile responsiveness.
Backend	Node.js, Express, TypeScript	Asynchronous, non-blocking I/O runtime optimal for RESTful micro-services and transaction processing; shared TypeScript interfaces between client and server prevent schema drift.

3. Functional Requirements

Each functional requirement is specified using the formal "The system shall ..." syntax and prioritized using the MoSCoW framework (Must, Should, Could). Key intelligent automation features are marked with **(AI)**.

ID	Requirement	Priority
FR-01	The system shall authenticate users via their mobile phone number, Azerbaijani FİN code, and a time-limited 6-digit SMS OTP verification code.	**Must**
FR-02	The system shall maintain and compute real-time account balances, earmarked goal reserves, unallocated discretionary cash, and net cash flow using exact two-decimal math.	**Must**
FR-03	The system shall dynamically calculate and display the daily savings contribution required for each active goal based on target amount, current balance, and target deadline.	**Must**
FR-04	The system shall enforce an anti-impulse cooling-off lock on savings goals, disallowing immediate withdrawals until a user-configured countdown timer (e.g., 30 minutes) expires.	**Must**
FR-05	The system shall connect to open banking sandbox providers (Kapital Bank, ABB, Leo, CBAR) and import incoming transactions while automatically deduplicating feeds.	**Must**
FR-06 (AI)	The system shall utilize an AI language model (Gemini 2.5) to analyze raw, unstructured bank transaction descriptions and automatically classify them into localized categories (Food, Transport, Housing, Utilities).	**Should**
FR-07 (AI)	The system shall generate context-aware AI financial advice and daily habit summaries by analyzing monthly net cash flow variances and saving consistency patterns.	**Should**
FR-08	The system shall track consecutive daily review streaks, award milestone badges (10, 20, 30, 40, and 100 days), and update user financial discipline experience levels.	**Should**
FR-09	The system shall generate comprehensive month-over-month comparative financial reports and allow authenticated export of transaction histories in CSV format.	**Could**
FR-10	The system shall allow users to manage their profile settings, switch between multiple demo workspaces, and toggle localized interface languages (Azerbaijani, English).	**Could**

4. Monitoring and Alerting

To guarantee high availability and early anomaly detection, Q□na□t incorporates an automated monitoring and alerting subsystem. The system continuously tracks: (1) backend service availability and HTTP response latencies; (2) server error spikes (HTTP 500 status rates); (3) suspicious authentication behavior such as repeated failed OTP or FİN attempts; (4) open banking connector disconnections or synchronization failures; and (5) AI inference timeouts.

How it works: (1) Operational events and performance telemetry are captured in real time through Express middleware, health check endpoints, and unhandled exception hooks; (2) Rule evaluators compare metrics against predefined failure thresholds; (3) When an anomaly or threshold violation is detected, an asynchronous dispatcher invokes the Telegram Bot API webhook; (4) The Telegram bot formats and transmits an urgent markdown alert directly to the team's dedicated engineering channel.

Secret Management: In compliance with security best practices, the Telegram bot token (`TELEGRAM_BOT_TOKEN`) and destination chat identifier (`TELEGRAM_CHAT_ID`) are strictly isolated inside a local `.env` file. This file is loaded at runtime through secure environment configuration and is explicitly registered in `.gitignore`, ensuring credentials are never committed to the public Git repository.

What we monitor	When it triggers an alert	Alert message
Backend Health Check	Health check endpoint (`/api/health`) fails 3 consecutive times in 60 seconds	**`[ALERT] Backend is DOWN! Health check failed 3 consecutive times on port :3000.`**

4. Monitoring and Alerting (Continued)

The table below details the full operational incident matrix, detection rules, and automated Telegram bot dispatch notifications:

What we monitor	When it triggers an alert	Alert message
Suspicious Auth & Brute-force	5 consecutive failed OTP codes or password attempts for a phone number or FİN within 5 minutes	`[ALERT] Potential Brute-force Attack: 5 failed authentication attempts for target account within 5m.`
Open Banking Sync Failure	External banking provider API (Kapital, ABB, Leo, CBAR) fails or drops connection 3 times in a row	`[WARNING] Open Banking Sync Failed: Bank provider API unreachable or returned invalid response schema.`
AI Inference Latency & Quota	Gemini AI transaction categorization takes > 4.5 seconds or returns rate limit / quota exhaustion	`[WARNING] AI Service Degradation: Gemini categorization latency exceeded 4.5s or returned quota error.`
API Response Time Latency	REST API average response latency exceeds 3.5 seconds across rolling 5-minute sampling window	`[WARNING] High API Latency: Endpoints averaging 3.8s response time over last 5 minutes.`
Database Persistence Failure	Atomic filesystem write fails, file lock contention occurs, or database corruption is detected	`[CRITICAL] Database Write Error: Atomic flush failed for finance_db.json. Disk write permission error.`
Cooling-off Safety Lock Tamper	Direct goal withdrawal attempt initiated without an active or expired unlock cooling-off request	`[WARNING] Security Breach Attempt: Direct withdrawal rejected on locked goal #ID without cooling-off timer.`

> **Course Project Alignment & Implementation Status:**
>
> All core architectural layers described in this proposal have been developed into an operational software system:
>
> - **Production-ready Frontend & Backend:** React 19 SPA running in synchrony with a TypeScript Express REST API on port 3000.
> - **Financial Calculations Engine:** Verified with exact 2-decimal math, dynamic countdown algorithms, and streak mechanics.
> - **Automated Testing & Browser Verification:** Fully validated using Playwright headless browser automation, confirming zero console regressions.
> - **Localized Fintech UX:** Bilingual support with native Azerbaijani financial terminology and AZN currency formatting.